DIVISION OF PHYSICAL SCIENCES AND MATHEMATICS
College of Arts and Sciences | University of the Philippines Visayas
Miagao, Iloilo, 5023, Philippines | Tel. No. (033) 513-7020
Email Address: psm.upvisayas@up.edu.ph

COURSE MATERIAL

The One That Got Away

CMSC 124 Problem Set 4

Prepared by: Rene Andre Bedonia Jocsing

Published: November 12, 2026 **Deadline:** November 25, 2026

This is our final battle.

Suppose you have four facts and two rules:

```
edge(a, b).
edge(b, c).
edge(c, a).
edge(c, d).

reach(X, Y) :- reach(X, Z), edge(Z, Y).
reach(X, Y) :- edge(X, Y).
```

Now ask whether `d` can be reached from `a`:

```
?- reach(a, d).
```

Which causes this trace:

```
reach(a,d)
  clause 1:  reach(a,Z1), edge(Z1,d)
    reach(a,Z1)
      clause 1:  reach(a,Z2), edge(Z2,Z1)
        reach(a,Z2)
          clause 1:  reach(a,Z3), edge(Z3,Z2)
            ...
```

As you can see it never answers. It doesn't answer `false`, which would be wrong. Nothing. The program sits there until you stop it.

Now swap the two rules, so that the one whose whole body is `edge(X, Y)` comes first, and the same question answers `true`:

```
reach(a,d)
  clause 1:  edge(a,d)          -> fail
  clause 2:  reach(a,Z), edge(Z,d)
    reach(a,Z)
      clause 1:  edge(a,Z)      -> Z=b
    edge(b,d)                   -> fail
    backtrack into reach(a,Z)
      clause 2:  reach(a,Z2), edge(Z2,Z)
        reach(a,Z2)
          clause 1:  edge(a,Z2) -> Z2=b
        edge(b,Z)               -> Z=c
    edge(c,d)                   -> true (hint: peek at line 3)
```

Same facts. Same rules. Same logic, in the sense a logician means it, because the two versions say the same thing about `reach`. The only difference is the order in which they say it.

That's the last idea of the course. It's the reason Prolog is here. Activity 15 built a runtime stack by hand. A frame for every active call, a return address in every frame, and a stack that unwinds when the deepest call returns. Prolog runs on that same machinery and adds one thing: When a call fails, the program goes back to the most recent decision it made and makes a different one. Which decision comes first depends on the order of the rules. In the example above, the first decision is one it never gets back from.

You'll do this in two halves. First you'll predict 16 queries before running any of them, three answers per query, 48 predictions in total, and write why each one comes out the way it does. Then you'll write seven small rules in Prolog, in three groups, ending with the one that walks the graph above and comes home.

This is a paired, asynchronous assignment. It runs on SWI-Prolog alone. One install command gets it. Most of you meet Prolog for the first time here, so Part 1 also tells you which book to read while you work. This manual carries the rules the 16 cases grade. The book teaches the language.

Why These Three Fields?

A Prolog program is a list of **facts** and **rules**, together called a **knowledge base**. `edge(a, b).` is a fact. It says the relation `edge` holds between `a` and `b`. That's all it says. A **rule** has a head and a body, `head :- body.`, and it says the head holds whenever every goal in the body does. A **query** is a question you type at the prompt, and Prolog answers it by searching the knowledge base for a way to make it true.

Take a query with a variable in it:

```
?- parent(tom, X).
```

A word that starts with a capital letter is a **variable**. Prolog looks for a fact or a rule head that matches `parent(tom, X)`, and matching is done by **unification**, which finds values for variables that make two terms identical. The first `parent` fact whose first argument is `tom` unifies, `X` becomes whatever its second argument is, and Prolog prints that binding.

That's one answer. It isn't the only one. The third field comes from what happens next. At the prompt, Prolog waits after an answer. Press `;` and it goes back to the last choice it made, undoes the binding, and looks for the next fact that unifies. That step is called **backtracking**. Keep pressing `;` and it keeps going until the facts run out, when it prints `false`.

So each case asks three questions:

Field	Question
`first`	What is the first answer Prolog prints, in the grader's notation?
`second`	What does the first `;` produce, or `none` when there is no second answer?
`count`	How many answers does the search find before it says `false`?

What an Answer Looks Like

An answer is a set of bindings, one for each variable in the query. The grader writes `parent(tom, X)`'s first answer as `X=liz`. A query with two variables gets two bindings, joined by a comma, in the order the variables appear in the query. A query with no variables has nothing to bind, so its answer is `true`. A query with no answer at all reads `false`.

Two more words cover the cases that don't end that way. When a search runs past the grader's budget, the count reads `unbounded`, and so does any answer the search hadn't reached when the budget ran out. When a query throws an error, all three fields read `error`. Two of the 16 cases are unbounded and one is an error. Each of them is there to teach you something.

The Search Has an Order

Two rules decide everything you predict here. Prolog tries clauses in the order they're written, top to bottom. Within a rule body, it tries goals in the order they're written, left to right. That's the whole scheduler.

Those two rules explain the opening example. `reach(a, d)` tries the first clause, whose body starts with `reach(a, Z)`. To answer that it tries the first clause again, whose body starts with `reach(a, Z)`. No fact is ever consulted. Nothing ever fails, so nothing ever backtracks. The stack grows until something outside the program stops it. Put the `edge` clause first and each call reads a fact before it recurses. The answer arrives at once. Whether the search then ends is a different question, and P12 asks it.

The goal order inside a body matters just as much. One of the 16 cases turns on it. `X > 2` can only be checked when `X` already holds a number. Ask it before anything has bound `X` and Prolog throws an error. Ask it after a goal that binds `X` and it works. Same goals, different order, and one of them is an `error` row in your table.

Learning Objectives

By the end of this problem set, you should be able to:

- Read a fact, a rule, and a query, and say what each one claims
- Unify two terms by hand, and say which variable ends up bound to what
- Predict the first answer to a query by following clause order and goal order, without running it
- Predict what backtracking finds next, and when it finds nothing
- Explain why swapping two clauses can change the order of the answers, and why in one case it changes whether the search ever answers
- Write a recursive rule with a base case that reads one fact and a recursive case that asks a smaller question
- Write a rule over lists, using the `[Head|Tail]` pattern
- Carry a visited list through a recursive walk so that a cyclic graph can't trap it
- Read a four-port trace of a search and match its lines to frames being pushed and popped on the runtime stack from Activity 15

The Pair

I assign pairs at random across every section. You don't choose your partner. You probably won't work with your laboratory partner. If the pooled roster has an odd number of students, one trio receives the same work.

The published pairing sheet is authoritative. Both members predict all 16 cases before comparing answers. Splitting the table in half defeats the part that asks you to follow every search. After the prediction commit you may divide the rules, but review each other's code before the final run because the responsibility for it belongs to both of you.

You won't always be free at the same hours. This is asynchronous work, so the repository is where you meet. Push small commits, say what you did in the message, and open an issue on the fork as soon as you're stuck. Replies can take until tomorrow. If your partner goes quiet for a couple of days, or the two of you can't find any overlapping time, tell me while there's still most of the window left. The commit history already records who did what. That record is your case. What I can't do is fix it the night before the deadline.

This belongs to the lecture component. The laboratory booking, progress-report, and defense rules don't apply. The lecture deadline is strict.

Prolog Setup

You need SWI-Prolog and Git. The syllabus specifies Prolog, and SWI-Prolog is the implementation that produced the published answers. Every command in this repository runs through `swipl` on the command line. SWI-Prolog and Git are the whole toolchain.

Windows

Install Git first, from PowerShell:

```
winget install --id Git.Git --exact
```

Git for Windows brings **Git Bash**, a terminal that understands the Unix commands this manual uses. Open it from the Start menu. Run every later command in this manual in Git Bash. The PowerShell lines in this section are the only ones that belong in a PowerShell window. PowerShell has no `chmod` for `./check.sh`, so a command pasted into the wrong window fails in a way that looks like a broken starter.

SWI-Prolog has a winget package. Still in PowerShell:

```
winget install --id SWI-Prolog.SWI-Prolog --exact
[Environment]::SetEnvironmentVariable('Path', [Environment]::GetEnvironmentVariable('Path',
  ↪  'User') + ';C:\Program Files\swipl\bin', 'User')
```

The installer puts `swipl.exe` in `C:\Program Files\swipl\bin` and doesn't add that folder to your `PATH`, so the second line does. Close Git Bash and open it again so it picks up the change.

On Windows, when you want the interactive prompt, the one where you press ; for the next answer, open it from PowerShell by typing `swipl`. Git Bash's terminal doesn't hand keystrokes to `swipl` the way the prompt needs. The scripts in the starter don't need the prompt, so they run in Git Bash like everything else.

Ubuntu

The Ubuntu repository carries SWI-Prolog:

```
sudo apt update
sudo apt install -y swi-prolog
```

macOS

Install SWI-Prolog with Homebrew:

```
brew install swi-prolog
```

Verify the Runtime

In Git Bash, or your terminal on Ubuntu and macOS, run:

```
swipl --version
```

It reports a version. Ubuntu's package is the 9.0 series and the other two are 10.0. All three produce the same 48 published answers. The grader checks that every time it runs. If the command isn't found, fix the installation before you clone the starter.

Now try the prompt once, so Day 2 isn't the first time you see it. On Ubuntu and macOS type `swipl` in your terminal. On Windows type `swipl` in PowerShell. You get a `?-` prompt. Type `X = 1.` after it, including the period:

```
?- X = 1.
```

It prints `X = 1.` Type `halt.` to leave. Every query ends with a period. Forgetting it is the first mistake everyone makes. The prompt waits for the rest of the line.

Get the Starter

Fork cmsc-124-ps4-starter and clone your fork. A fork is a snapshot. A fix I push to the starter mid-window never reaches it, so what you were graded against can't change under you.

```
git clone https://github.com/<your-account>/cmsc-124-ps4-starter.git
cd cmsc-124-ps4-starter
chmod +x run trace lint check.sh
./check.sh
```

One of you forks the repository under your account, keeps the fork public, and adds the other member as a collaborator, since the fork is the pair's deliverable. Then open the Actions tab on the fork and enable workflows. GitHub disables them on every fork, so that tab shows a short notice and a button. GitHub documents the rule and the fix under Workflows in forked repositories. The workflow run is a deliverable, so don't discover this on Day 11.

A fresh starter reports `1/67 checks passed` and exits 1. That red run is the correct starting state. Forty-eight checks belong to the prediction table, sixteen to `src/rules.pl`, one to the load of that file, one to `REASONING.md`, and one to `ANALYSIS.md`. The analysis check stays failed until `ANALYSIS.md` lands inside the 300 to 450 word range with the placeholder gone.

The one starting pass is `rules_load`, which asks only whether `src/rules.pl` consults without a syntax error. The stubs manage that on the first commit. A stub that throws when called is still valid Prolog. Read `1/67` as the starting state. Nothing has been done yet.

The Repository

```
cases/kb.pl                the facts and given rules every query runs against
cases/cases.pl             the 16 queries, as you would type them at the prompt
predictions.tsv            your three predictions for every query
REASONING.md               why each case gives its values, written before you run
src/rules.pl               the seven predicates you implement
tests/expected.tsv         every published expected result
tests/check_all.pl         the complete public automated checker
scripts/support.pl         the notation and the answer enumerator
scripts/run_case.pl        the one-case runner
scripts/trace_case.pl      the tracer
scripts/validate_predictions.pl  the prediction table's form checks
scripts/lint_predictions.pl      what ./lint runs
run  trace  lint  check.sh       the course run contract
ANALYSIS.md                the final explanation you write together
.github/workflows/test.yml  your CI run of the same checks
```

You edit four files: `predictions.tsv`, `REASONING.md`, `src/rules.pl`, and `ANALYSIS.md`. You also replace the starter's `README.md` with your own. Keep the knowledge base, the case corpus, the expected table, the runner, the tracer, the grader, and the `scripts/` helpers unchanged. The grader runs all 16 queries before scoring and stops if `swipl` disagrees with the published expected table.

`check.sh` is the complete public automated check for the predictions, the rules, and the analysis word range. The same script runs on your machine, in your GitHub Actions workflow, and during grading. There's no hidden test suite. The rubric also judges the analysis itself, the commit history, and the submitted workflow runs. A `67/67` result satisfies the local test requirement. Your final grade still depends on those rubric rows.

Your repository being a fork makes the fixture check cheap. Grading starts from the starter tag and reads the fork's diff against it, so an edited fixture is visible as a diff in the open.

That policy creates an awkward fact for a prediction assignment. The public file `tests/expected.tsv` contains the answers, because grading can't depend on secret results. Don't open it until your prediction commit exists. Git records when your table changed. It can't prove which files you read first, so this part depends on your honesty.

The Knowledge Base Has No Cut

If you've read ahead, you may have met `!`, the **cut**, and `\+`, negation. `cases/kb.pl` uses neither, and it says so at the top. The tracer behind `./trace` walks the knowledge base one clause at a time and reports every step. A cut lets a clause skip steps the tracer can't see. Keeping the base plain keeps the trace honest. Your own rules in Part 2 may use `\+`. The last group needs it. They're graded on their answers and never traced.

Part 1: Predict Before Execution

Open `cases/kb.pl`, `cases/cases.pl`, and `predictions.tsv` side by side. The prediction file starts like this:

```
id  first   second  count
P01 TODO    TODO    TODO
```

The case file looks like this:

```
ps4_case("P01", "parent(tom, X)").
```

Each case is stored as the text you'd type at the prompt, minus the `?-` and the period. That's why the variable names you see there are the names the grader prints back. `parent(tom, X)` answers `X=liz`, not `_123=liz`.

Read the rules below and the notation after them. Fill the table, and write one entry per case in `REASONING.md` that names the rule behind each field. Commit both, then run anything.

The Rules You Need

A **term** is anything Prolog can talk about. `tom` is an **atom**, a name that stands for itself. `3` is a number. `X` is a variable. `parent(tom, X)` and `f(Y, a)` are **compound terms**, a name followed by arguments in parentheses. `[a, b, c]` is a list, which is a compound term wearing a convenient costume.

Unification makes two terms identical by binding variables, when that's possible. `f(X, b)` unifies with `f(a, Y)` by binding `X` to `a` and `Y` to `b`. `a` doesn't unify with `b`, because two different atoms are different and no binding can change that. `X = f(Y, a)` doesn't compute anything. It unifies `X` with the term `f(Y, a)`, leaving `Y` inside it unbound, and a later `Y = b` fills that in. The `=` you see in three of the cases is unification and nothing else.

A **query** is answered by finding a clause whose head unifies with it. For a fact, that's the whole job. For a rule, the goals in the body become new queries, tried left to right. When every goal in the body succeeds, the head succeeds with whatever bindings the body made. When a goal fails, Prolog backtracks to the most recent goal that still has untried clauses, undoes the bindings made since, and tries the next one.

That machinery decides how `grandparent(tom, X)` plays out. The rule is `grandparent(G, C) :- parent(G, P), parent(P, C).` Prolog binds `P` to tom's first child, then asks whether that child has a child. If not, it backtracks, rebinds `P` to tom's second child, and asks again. The facts are written in an order that makes this happen, and `./trace P07` shows it.

Recursion in Prolog is a rule that calls itself with a smaller question. `ancestor(A, D)` holds when `parent(A, D)` does, and also when `A` is a parent of someone who is an ancestor of `D`. The base clause reads one fact. The recursive clause reads one fact and then asks a smaller question. Which clause comes first decides the order the answers come out in, and P08 and P09 are the same question asked of two clause orders.

A **list** is written `[a, b, c]`. The pattern `[H|T]` unifies with any nonempty list, binding `H` to its first element and `T` to the rest. `[]` is the empty list and matches nothing else. Every list rule you'll meet has a clause for `[]` and a clause for `[H|T]`, which is the base case and the recursive case again.

Arithmetic needs `is`. `N is M + 1` computes the right side and unifies the result with `N`. `N = M + 1` would unify `N` with the compound term `M + 1`. No number would ever appear. Both `is` and the comparison `>` need their operands bound to numbers already. Ask `X > 2` while `X` is unbound and Prolog throws an instantiation error, which is a precise way of saying "you asked before you told me."

Required Reading

The 16 cases grade those rules. This manual covers those rules. Teaching the language from nothing takes a tutorial longer than the window. You'd still be reading it on Day 11. So read the book below as you work.

Read *Learn Prolog Now!* by Patrick Blackburn, Johan Bos, and Kristina Striegnitz. The complete text is online for free. This edition lets you run its examples in the browser:

```
https://lpn.swi-prolog.org/lpnpage.php?pageid=online
```

Five of its chapters carry this problem set:

What you're working on	Where to read it
Facts, rules, queries, and what a variable is	1. Facts, Rules, and Queries
How two terms unify, and how the search picks clauses and goals	2. Unification and Proof Search
Base case, recursive case, and why clause order matters	3. Recursion
`[H|T]` and the rules over lists in Part 2	4. Lists
`is`, `>`, and why an unbound operand is an error	5. Arithmetic

Chapters 1 and 2 in that order cover most of what the 16 cases ask. Read chapter 3 before you predict P08 through P12, and chapters 4 and 5 before P13 through P16. Read chapters 4 and 5 again the evening before Part 2. The last group in Part 2 uses `\+`, which chapter 10 covers under negation. Read the negation section when you get there. The rest of that chapter covers the cut. You don't need it.

Two more pages to keep open. SWI-Prolog's manual describes the four-port model that `./trace` prints, in section 2.10.1, "The Byrd Box Model And Ports":

```
https://www.swi-prolog.org/pldoc/man?section=debugoverview
```

And the prompt itself is a reference. `swipl cases/kb.pl` opens a prompt with the knowledge base loaded. Any query you can type there, you can press `;` on. That's the cheapest way to check a rule you're unsure of, on a query that isn't one of the 16.

Grader Notation

The `first` and `second` fields are one line of text each describing an answer. The grader compares them letter for letter against what `swipl` produces, so the notation is part of the answer. Below is the whole notation, which is more than the 16 cases need.

Prolog answer	Written as
`true`	`true`
`false`	`false`
`X = liz`	`X=liz`
`X = liz, Y = bob`	`X=liz,Y=bob`
`X = f(b, a)`	`X=f(b,a)`
`X = [1, 2]`	`X=[1,2]`
`X = []`	`X=[]`
`N = 3`	`N=3`
a variable the answer left unbound	`_`
the search ran past the budget before this answer	`unbounded`
the query threw an error	`error`

Three mechanical rules go with the table. Elements and bindings are separated by commas and nothing else. No field contains a space anywhere, so `X = liz` fails a check that `X=liz` passes, and a cell padded to line up your columns in an editor fails for the same reason. Bindings appear in the order the variables first appear in the query text, which is the order the prompt prints them in.

The prompt prints spaces after commas and around =. You don't. That's the only difference between what you see at the prompt and what you write in the table.

The `count` field is a whole number, or `unbounded`, or `error`. The `second` field is an answer in the notation above, or `none` when the search finds fewer than two.

If you prefer the rule to the table, `scripts/support.pl` is the code that writes these descriptions. It's short. The grader, `./run`, and `./trace` all call it, so nothing else decides what a description looks like.

Predict, Then Commit

Replace every `TODO` in `predictions.tsv`. Keep the tab-separated layout and the case order. Then check the table's form:

```
./lint
```

`./lint` reads the form only. It never opens `tests/expected.tsv` and never runs a case, so it tells you nothing about whether an answer is right, and that's what makes it safe to run before the prediction commit. What it catches is the class of mistake that costs marks for no reason. A padded cell, a space inside an answer, a count that isn't a number. Those all survive the parse and then fail their comparisons, so they read like wrong predictions. Others are worse. A tab your editor replaced with spaces, or a missing or reordered row, stops the grader before it scores anything. One trailing space on every answer row fails all 16 `count` checks. Each failure prints as `FAIL P01.count`, a label that never mentions a space.

It reports each fault with its line and column:

```
line 3, column count: "one" is neither a whole number, unbounded, nor error. A
search that ends has 0 or more answers, one that never ends reads unbounded,
and one that throws reads error.
```

Line 3 is P02. The header is line 1.

Now the reasoning. A right answer can be copied from the table, and `./run` prints every one of them, so the answers alone can't show the work. The grade turns on why each case comes out the way it does. That's what `REASONING.md` holds: one entry per case, written before you run anything, naming the rule that fixes each field. The starter ships the file with a heading for every case and one placeholder line under each. Replace each placeholder with one to three sentences.

The grader checks the structure of the file, not the prose. It wants every heading `## P01` to `## P16` in order, a non-blank line under each, and the placeholder line gone. There's no word minimum, because a minimum invites padding. What it can't see is whether your reasoning is any good. That part is a rubric call, and it rewards an entry that names a rule over one that restates the answer.

An entry looks like this. The example case isn't one of the 16, so it gives nothing away:

```
## parent(pat, X)

The goal scans the parent facts in source order and takes the first whose
first argument is pat. Only parent(pat, jim) matches, so X binds to jim.
No fact is left, so the first ; finds nothing and the count is 1.
```

When it says the table is well formed and complete, commit the table and the reasoning together, before running any case:

```
git add predictions.tsv REASONING.md
git commit -m "docs: predict Prolog answers and counts"
```

That commit records your prediction before you inspect a case or open `tests/expected.tsv`. Git stamps when `REASONING.md` first appeared, so reasoning added after you run the cases doesn't read as a prediction. Write it first, because a later entry doesn't carry the grade.

After the commit exists, you may run:

```
./run P01
```

The runner prints one description:

```
case: P01
first: X=liz
second: X=bob
count: 2
```

Compare the observation with your row. When they differ, keep the prediction commit and correct the table in a later commit. Say which rule you got wrong in that commit message:

```
git add predictions.tsv
git commit -m "fix: the recursive clause first changes the answer order"
```

Add a correction entry to `REASONING.md` under every case you got wrong, in the same commit or the next one. It records what you predicted, what the language printed, and which rule you misread:

```
## Correction for parent(bob, X)

I predicted X=pat first. The prompt printed X=ann first, because parent(bob,
ann) is the fact written above parent(bob, pat). I read the facts as sorted
by the child's name instead of by their order in the file.
```

That history is more useful than a table made perfect after the answers were already on screen. The rubric reads the reasoning and the corrections, so a correct table with no reasoning in the prediction commit earns no better than Fair.

Watch the Search

When a row surprises you, don't reread the rules. Watch the search:

```
./trace P07
```

```
== trace P07 ==
Call: grandparent(tom,X)
  Call: parent(tom,_)
  Exit: parent(tom,liz)
  Call: parent(liz,X)
```

```
  Fail: parent(liz,X)
  Redo: parent(tom,liz)
  Exit: parent(tom,bob)
  Call: parent(bob,X)
  Exit: parent(bob,ann)
Exit: grandparent(tom,ann)
answer 1: X=ann
Redo: grandparent(tom,ann)
  Redo: parent(bob,ann)
  Exit: parent(bob,pat)
Exit: grandparent(tom,pat)
answer 2: X=pat
Redo: grandparent(tom,pat)
  Redo: parent(bob,pat)
  Fail: parent(bob,X)
  Redo: parent(tom,bob)
  Fail: parent(tom,_)
Fail: grandparent(tom,X)
== 2 answers ==
```

Every goal in a search passes through four doors. The trace prints one line each time it goes through one. `Call` is the goal being tried for the first time. `Exit` is the goal succeeding, with the bindings it made shown in place. `Redo` is the search coming back to the goal after something later failed, to try its next clause. The record it comes back to, which clause matched and that more remain, is a **choice point**. Activity 15's returned frames kept no such record. `Fail` is the goal running out of clauses. Indentation is depth. A goal's body sits one level below it. These are the same four words that `swipl`'s own debugger prints, with the keystrokes removed.

Read the trace above once with the rule for `grandparent` beside it. `P` binds to `liz` first because that's the first `parent(tom, _)` fact. `liz` has no children, so `parent(liz, X)` fails. The search goes back through `Redo` to the fact that bound `liz`, undoes that binding, and takes the next fact. That's the whole of backtracking, in six lines.

Two of the cases stop the tracer at 400 lines with a message. That's the tracer giving up, not the search ending. Read the goal that repeats and you'll see why it won't.

Part 2: Write the Rules

Now you write rules of your own, against the same facts.

`src/rules.pl` is loaded after `cases/kb.pl`, so `parent/2`, `edge/2`, `elem/2`, `join/3`, and `len/2` are yours to call. The notation `name/2` means a predicate called `name` that takes two arguments. `elem(X, L)` holds when `X` is an element of `L`. `join(A, B, C)` holds when `C` is `A` followed by `B`. `len(L, N)` holds when `L` has `N` elements. All three are defined in the knowledge base. P13, P14, and P16 trace them.

The grader scores three groups independently. A pair that finishes R1 and R2 and never gets `route/3` to end keeps every mark from R1 and R2. Work in order.

Each stub in the file throws when called, so a check you haven't reached fails with a message that says which predicate is missing. Replace the whole stub, head and body, with your own clauses. A stub left above your clauses runs first and throws before yours are ever tried.

R1: The Family

R1 is three rules over `parent/2`.

`sibling(X, Y)` holds when `X` and `Y` share a parent and aren't the same person. The second half is `X \= Y`, which succeeds when its two sides can't be unified. `X \= Y` with either side unbound fails

at once because an unbound variable unifies with anything. Put it before the goal that binds them and `sibling(ann, X)` answers nothing. Put it after and it works. One of the six R1 checks asks that query.

`cousin(X, Y)` holds when a parent of `X` and a parent of `Y` are siblings. Use the rule you just wrote.

`descendant(D, A)` holds when `A` is an ancestor of `D`. Write it as the mirror image of `ancestor/2` in `cases/kb.pl`, two clauses, base first. The grader checks the set of descendants and the set of ancestors and ignores their order, so a different definition that finds the same set also passes. The suggested one is the one you'll be able to follow by hand, one clause at a time.

```
?- sibling(ann, X).
X = pat.

?- cousin(sue, X).
X = jim ;
false.

?- descendant(sue, X).
X = ann ;
X = bob ;
X = tom ;
false.
```

R2: Lists

Three rules over lists, each with a base case and a recursive case.

`count_of(X, List, N)` holds when `X` occurs `N` times in `List`. The empty list holds zero of anything. That's your base case. For a nonempty list, the head either is `X` or isn't, so the count is the tail's count plus one, or the tail's count unchanged. Write those as two clauses. The one for "isn't" needs `X \= H`, and by now you know where to put it.

`rev(List, Reversed)` holds when `Reversed` holds the elements of `List` in the other order. Reverse the tail, then use `join/3` to put the head on the end. This is the classic naive reverse. It has a property you should see once. Run it backward:

```
?- rev(X, [1, 2]).
X = [2, 1] ;
```

The first answer is right. Press `;` and it never comes back. The recursive call `rev(T, RT)` has both arguments unbound, so it enumerates lists of every length looking for one whose reverse is `[1, 2]`, and there's no length at which it can stop. The grader asks for the first answer only, under a budget. Your `rev` is correct if it finds `[2, 1]` first. It isn't wrong for looping afterward. That's the definition doing what it says.

`last_of(List, X)` holds when `X` is the last element of `List`. A one-element list's last element is that element. The empty list has no last element, so `last_of([], X)` must fail, and it does on its own if you don't write a clause for `[]`.

R3: The Graph

One rule, the one the opening example was about.

`route(From, To, Path)` holds when `Path` is the list of nodes from `From` to `To`, in order, following `edge/2` and never visiting a node twice. A route from a node to itself is `[From]`.

```
?- route(a, d, P).
P = [a, b, c, d] ;
```

```
false.

?- route(c, b, P).
P = [c, a, b] ;
false.
```

The graph has a cycle, `a` to `b` to `c` and back to `a`. `reach_r/2` in the knowledge base walks this graph without remembering where it's been, and P12 shows what that costs. It finds `true`. Press `;` and it goes round the cycle, finds `true` again, and never stops finding it. That's why P12's count is `unbounded`. `route/3` has to be the version that comes home.

The standard way is a helper that carries the visited list, something like `travel(From, To, Visited, Path)`. `Visited` holds every node seen so far. The base case is reaching `To`. The recursive case takes one edge to a `Next`, refuses it when `Next` is already in `Visited`, and continues from there with `Next` added. "Refuses it" is where `\+` comes in. `\+ Goal` succeeds when `Goal` can't be proved. So `\+ elem(Next, Visited)` says "not yet visited." Chapter 10 of the book explains why this is called negation as failure. It also explains why it's only safe when `Next` is already bound. It's here, since `edge(From, Next)` ran first.

If you build `Visited` by adding to the front, the finished list comes out backward. You have a `rev/2` from R2.

Each query that a check in R1, R2, and R3 asks runs under a budget of 200,000 inferences. The model solution spends under 150 on any one query. A `route/3` that goes round the cycle fails those checks with a message that states the budget, and so does a rule that repeats its work thousands of times over. Your run doesn't hang.

Part 3: Explain the Search

Replace the placeholder in `ANALYSIS.md` with 300 to 450 words written by both partners, plus one pasted trace. The grader counts every space-separated token in the file, so a heading counts too, and it skips the lines a `./trace` produces, so the trace costs you no words. It fails while the placeholder text is still there. It fails again when the total lands under 300 or over 450. It enforces both limits. Aim for the middle, so one last round of editing leaves you inside the range.

Paste the output of `./trace` for one case, then answer these:

- Which lines in your trace are a frame being pushed onto the runtime stack from Activity 15, and which are a frame being popped?
- A `Redo` line resumes a goal that already exited. What does that goal still know that a returned frame in Activity 15 had already thrown away?
- Swapping the two clauses of `reach/2` changes whether the search ever answers. Swapping the two clauses of `ancestor/2` only changes the order of the answers. What's the difference between the two recursive clauses that makes that so?

Use your trace as evidence. Point at its lines. A general description of backtracking doesn't answer the prompt. Neither does a paraphrase of this manual.

Running the Checks

Run one case while working on predictions:

```
./run P10
```

Watch one search:

```
./trace P10
```

Run the whole public grader after every coherent change:

```
./check.sh
```

A complete submission ends with:

```
== result ==
67/67 checks passed
```

`67/67` is the complete automated result. The rubric separately assesses your explanation, your commit history, and your workflow runs.

Check the table's form at any time, which reveals no answers:

```
./lint
```

`./run` and `./trace` exit 0 when they print the case, 64 when called with the wrong number of arguments, and 65 when the case id is unknown. `./lint` exits 0 when the table is well formed and complete, and 1 otherwise. `./check.sh` lists the same form faults before it scores, under a `== predictions.tsv form ==` heading. Read that heading first when it appears. A fault under that heading is a typing mistake.

`check.sh` exits 0 only when all 67 checks pass. An error inside one of your rules becomes a labeled failed check while the script keeps going. The message prints under the failure. Start at the first one.

Each rule group has its own checks. Delete `route/3`. R3's four checks fail while R1 and R2 stay at full marks. A missing predicate fails its own checks and the checks of any rule of yours that calls it. If your `route/3` calls `rev/2`, deleting `rev/2` fails R3 as well.

One fault in the table's layout stops the grader before it scores anything. The prediction table has to keep its four columns and all 16 ids in order. When it doesn't, the script says so and quits. So a run that ends without a `== result ==` line points at the layout of `predictions.tsv`. A stray tab, a deleted row, or an editor that turned the tabs into spaces explains it.

The supplied workflow runs the same commands on Ubuntu, macOS, and Windows. Push early enough to see all three jobs finish. A local green run is required. The three workflow jobs must also pass from a clean checkout.

Milestones

Of the twelve days in the window, this schedule uses eleven, which reserves the last day for verification. Start on Day 1.

Day	Date	Work
1	Sat Nov 14	Install SWI-Prolog, clone the starter, read the knowledge base and all 16 queries together, start the reading
2	Sun Nov 15	Fill the table and `REASONING.md` independently, compare answers, commit both before running anything
3	Mon Nov 16	Run P01 to P08. Trace the ones you got wrong. Add correction entries. Free lecture session
4	Tue Nov 17	Run P09 to P16. Finish the prediction checks and their corrections
5	Wed Nov 18	R1. `sibling` and `cousin`, and the place `\=` goes
6	Thu Nov 19	R1. `descendant`. Free lecture session
7	Fri Nov 20	R2. `count_of` and `last_of`
8	Sat Nov 21	R2. `rev`, in both directions
9	Sun Nov 22	R3. `route`, until the walk comes home
10	Mon Nov 23	Write `ANALYSIS.md` together. Free lecture session
11	Tue Nov 24	Run `./check.sh`, push, read all three workflow jobs, submit

Your laboratory work overlaps this window. The Lab 4 defense window runs November 16 to 22. Lab 5's week 2 report window runs the same seven days. Its week 3 window opens November 23. Days 5 through 9 above are the ones with room to move, so if the laboratory takes a weekend, take it from there and not from the end.

Deliverables

Submit one public GitHub repository for the pair containing:

1. A completed `predictions.tsv`
2. A completed `REASONING.md`, committed before any case ran
3. The seven completed predicates in `src/rules.pl`
4. A 300 to 450 word `ANALYSIS.md` with one pasted trace
5. The unchanged knowledge base, cases, expectations, runner, tracer, grader, `scripts/` helpers, and workflow
6. A green `.github/workflows/test.yml` run on the submitted commit
7. A `README.md` that identifies both members and their GitHub usernames, replacing the starter's own

Both members must commit. Use the syllabus's semantic commit prefixes. Commit history should show the prediction step before execution-based corrections, and show each member's implementation or review work.

After the final push, open the repository's commit history on GitHub and copy the full hash of the commit you want graded. One designated member emails both items included in the body in this form:

```
Repository: https://github.com/<your-account>/cmsc-124-ps4-starter
Commit: <full commit hash copied from GitHub>
```

The submitted hash pins the version I grade. The green workflow run must belong to that commit. A later commit doesn't change the submitted version. CC the rest of the pair. Use this subject line, joining each person's entry with `&`:

```
[CMSC 124 Lec] PS4 Group: LastName1, Initials1 & LastName2, Initials2
```

A trio adds its third member in the same format.

Each member also emails `reflection.txt` individually. Identify the work you did, one search your prediction got wrong, one part your partner explained, and what you learned. Use:

```
[CMSC 124 Lec] PS4: LastName, Initials
```

For example, for me and my partner: `[CMSC 124 Lec] PS4 Group: Sanchez, SM & Jocsing, RA` as a group and `[CMSC 124 Lec] PS4: Sanchez, SM` individually for her.

Common Pitfalls

- Forgetting the period. Every clause in a file and every query at the prompt ends with one. Without it, the prompt waits and the file won't load.
- Reading `=` as assignment. It's unification. `X = f(Y, a)` builds nothing and computes nothing. It makes `X` and `f(Y, a)` the same term.
- Using `=` where you meant `is`. `N = M + 1` binds `N` to a compound term named `+`. Only `is` does arithmetic.
- Asking `>` or `is` before the operands are bound. That's an instantiation error, and one of the 16 cases is built to show it.
- Putting `X \= Y` before the goals that bind `X` and `Y`. It fails on the spot, and `sibling(ann, X)` answers nothing.
- Predicting an order without following the clauses. `ancestor` and `ancestor_r` find the same six people. Their first answers differ.
- Predicting `false` for a search that never ends. The search doesn't say `false`. It doesn't say anything, and the table says `unbounded`.
- Writing an answer with spaces in it. The comparison is exact, so `X = liz` is a wrong answer to a case whose result is `X=liz`. Run `./lint` and it prints the line and column of each fault before a check ever scores it.
- Walking the graph without a visited list. Every R3 check runs under a budget and tells you when the walk didn't come home.
- Leaving a stub above your own clauses. The stub runs first and throws.
- Running the cases before committing predictions. The Git history would no longer establish which answers were predictions.
- Leaving the placeholder line under a case in `REASONING.md`. The file has to carry the reasoning, not the template. The grader names the case it finds.
- Writing the reasoning after you run. The prediction commit is where it counts, and Git shows when each entry first appeared.
- Editing `tests/expected.tsv` until the checks pass. The published copy replaces it during grading.

Academic Honesty

Learn Prolog's rules from the required reading, the SWI-Prolog manual, and course material. That study is the assignment working as intended. You may also discuss those rules with your assigned partner. The prediction commit, later correction commits, rules, and analysis must reflect your pair's own reasoning.

Large language models may explain a language feature, but submitting generated predictions, code, or analysis as your own work is cheating. Each partner must be able to trace any of the 16 queries and explain any of the seven rules without a tool's help. The public answer table means this part relies on an honor rule. Copying it into `predictions.tsv` before doing the work is still cheating even though the file is available to you on purpose.

`REASONING.md` must land in the prediction commit, before you run a case or open `tests/expected.tsv`. Reasoning written after you run isn't a prediction. Git records when each entry first appeared, and the rubric reads that history. A correction entry added later is expected, and it belongs in the correction commits where it's visible as one.

Important Dates

Event	Date
Problem set released	Friday, November 13, 2026
Pairing sheet published	Friday, November 13, 2026
Submission deadline	Wednesday, November 25, 2026

The three lecture slots inside that window, November 16, 19, and 23, are yours to work in.

Lecture deadlines are strict. The laboratory's prescribed timeline and booking system don't apply.

Grading Rubric

Criterion	Excellent (90-100%)	Good (75-89%)	Fair (60-74%)	Poor (0-59%)
Predictions and Reasoning (20%)	Every case has a prediction-commit reasoning entry naming the rule behind each field. Every miss has a correction naming the misread rule. All 48 published first, second, and count checks pass by the final commit.	Every case has a prediction-commit reasoning entry. A few entries restate the answer without a rule, or a few misses lack a correction.	Many entries are generic or restate the answer, or many misses lack a correction. A correct table with no prediction-commit reasoning earns no more than Fair.	`REASONING.md` is missing or empty, or predictions and reasoning first appear after execution.
Family and Lists, R1 to R2 (20%)	All 12 checks for `sibling`, `cousin`, `descendant`, `count_of`, `rev`, and `last_of` pass. `\=` follows the goals that bind its operands, and each list rule has a base case and a recursive case.	At least 10 pass. One rule is off, such as a misplaced `\=` or a `rev` that never finds its first backward answer.	At least 6 pass. The family rules work but list recursion is unreliable, or the reverse.	Fewer than 6 pass, or the file doesn't load.
The Route, R3 (20%)	All 4 checks pass. `route/3` carries a visited list, finds `[a,b,c,d]` first, enumerates every route from `a` within the budget, and fails cleanly where no route exists.	At least 3 pass. The walk comes home, but one route is out of order or one dead end isn't handled.	Two checks pass. The cycle route and the dead end work, but the route to `d` and the enumeration from `a` exceed the budget, because the walk has no visited list or checks it after stepping.	Fewer than 2 pass, or `route/3` is unimplemented.
Analysis (20%)	The 300 to 450 word explanation pastes one trace, points at the lines that push and pop a frame, says what a choice point keeps that a returned frame doesn't, and explains both clause swaps by what each recursive clause consumes before it recurses.	All three questions are answered, with a thinner link to the pasted trace or a weaker account of the clause swaps.	The response describes backtracking in general but doesn't connect it to the pasted trace or to Activity 15's stack.	Missing, outside the word range, no trace pasted, or unable to say what a `Redo` line does.
Collaboration and Reproducibility (20%)	Both members make atomic semantic commits. `./check.sh` reports 67/67 locally and the submitted commit is green on Ubuntu, macOS, and Windows.	Both members contribute and the public checks pass, with one workflow or history weakness.	Contributions are markedly uneven, or the final clean-checkout workflow is red despite a mostly working local submission.	One member did the work, the grader was altered, or the submission can't run through `./check.sh`.